

The Oracle's Fractals

Gage Siebert, May 12th 2025

1 Introduction

This is a story about some objects that can only be seen in your imagination. In a way, these objects contain every possible computation. I will try to put an image of them into your mind without assuming much prior knowledge.

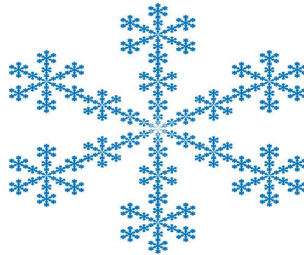


Figure 1: A run of the mill fractal.

Most fractals, like the one shown in Figure 1, share a couple interesting properties. First, no matter how far you zoom in on that fractal, it will still show structure- it will still look rough. Second, if you know where to look, you can find small exact replicas of the entire fractal inside itself. The objects we will discuss here have both of these properties, so they are fractals too.

Since fractals have details on infinitely small scales, there's a sense in which we can only ever draw them approximately. However, we can draw most fractals (including our example in Figure 1) so well, that we arrive at a point where if we were to improve upon the drawing any further, our eyes would not be able to tell. The same is true of a circle, and since we can draw all these shapes so well, it's reasonable to think that we *know* what they look like. However, for the fractals we will talk about later, even the rough outline of their shape is determined by unknowable truths.

In ancient myths, oracles are beings that dispense divine knowledge. In computer science, oracles are hypothetical entities that get used in proofs involving unknowability. For example, if we have a proof for some proposition, X , that says “it is unknowable if X is true or false”. Then we might write some proof about proposition Y 's knowability as follows: “If an oracle gave me the truth value of Y , I could use it to find the truth value of X . However, since X is unknowable, Y must be unknowable too.” Such unknowable truths are contained in the objects we will describe below. That is why you can only see them in your imagination, and that is why we will call them *The Oracle's Fractals*.

2 Preliminaries

This section will introduce some ideas we will need later to understand the Oracle's Fractals.

2.1 Strings

We will use Σ to denote a set of symbols that we are using as an alphabet. The binary alphabet, $\Sigma = \{0, 1\}$, is the simplest and most common case, where our possible symbols are just '0' and '1'. A string is a sequence of symbols. We will write Σ^* to denote the set of all possible strings written from our alphabet, Σ , including the empty string. We will use $\cdot$ to describe joining or concatenating two strings. For example, if $x = 011$ and $y = 000$, then $x \cdot y = 011000$. We will use $||$ to denote string length, so using x from before, we have $|x| = 3$.

2.2 Turing Machines

Turing machines are an abstract way to represent computation. They can be thought of in two parts: the tape and the head. The tape is infinitely long, and stores a linear sequence of symbols from some alphabet, Σ . We can always represent the state of the tape by a string from Σ^* ; this string is the sequence of symbols that are printed on the tape. The head, has a finite set of internal states, it can move up and down the tape, it can read the one symbol directly below it on the tape, and it can write on/erase that space of the tape (each space can only hold one symbol). The internal states of the Turing machine are instructions that, depending upon the symbol being read from the tape, they say if the Turing machine should move left or right over the tape, if/what it should write on the tape, and what the next internal state of the Turing machine is. There is a special internal state called the halting state. To perform a computation, a Turing machine is started on a tape. The string of symbols printed on the tape at the start is called the input program. The Turing machine then runs until its internal state becomes the halting state. At this point the state of the tape is another string which we call the output string. Turing showed that such a mechanism is capable of performing any formal deductive process, in other words, it can perform any computation.

For our purposes, it will suffice to represent a Turing machine, U , as a function from input strings to output strings, $U : \Sigma^* \rightarrow \Sigma^*$. We read $U(x) = y$ as 'Turing machine U produces output string y when given input string x .' We should be careful here. I am not saying every Turing machine is capable of producing every output string nor that they map every input to an output. Often, there are many inputs for which a Turing machine will never halt. Turing showed that being able to predict if a machine will halt is generally impossible, and this has formed the foundation for the modern notion of uncomputability.

2.3 Universality and Prefixes

We call a Turing machine *universal* if it is capable of simulating any other Turing machine. Formally, we say machine U_1 is able to simulate machine U_2 , if there is some prefix string, p , we can give U_1 that causes it to behave as though it were U_2 ; in other words $U_1(p \cdot x) = U_2(x)$ for every string x .

A countably infinite number of universal Turing machines exist, and we will write the set of them as $\{U_i\}$. From now on, unless otherwise stated, assume every Turing machine discussed is universal.

3 Building a Fractal

We are going to draw a picture with every output from a universal Turing machine. Input program strings will determine the part of the shape being colored, and the output strings will determine what colors are placed there. Our shape will live on a part of the real number line: $[0, 1)$, or the range from 0 to 1, including 0 but not 1.

We can use strings as an address system for the interval $[0, 1)$. Let's use the binary alphabet, so $\Sigma = \{0, 1\}$. Let s be a finite binary string. Now we can map s to a region in $[0, 1)$ as follows: if the first digit of s is 0, then we discard the second half of our interval. Otherwise if the first digit of s is 1, then we discard the first half of our interval. Our new interval is half the original, and we subdivide it again in the same way based on the second digit of s . We continue this process with progressively smaller intervals and later digits of s . Eventually, we reach the end of s and we are left with a sub-interval of $[0, 1)$ with size equal to $2^{-|s|}$. We will write $[s]$ to denote this region¹. This scheme is depicted visually in Figure 2.

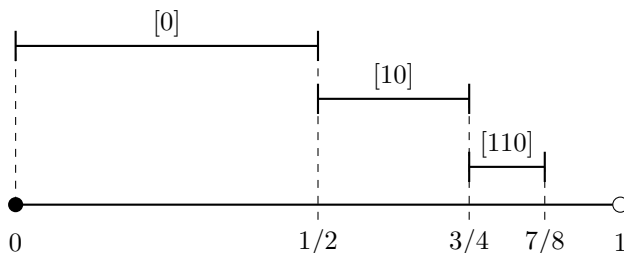


Figure 2: Three example sub-intervals of $[0, 1)$. We have by the process described above, $[0] = [0, 1/2)$, $[10] = [1/2, 3/4)$, and $[110] = [3/4, 7/8)$.

In the limit of the length of our string going to infinity, this interval becomes a point. Since our Turing machines' tapes are infinitely long, each specific program corresponds to a point in $[0, 1)$, and it will be finite prefixes that correspond to

¹If we used some other alphabet, then region size would scale as $|\Sigma|^{-|s|}$.

a region (a region whose points correspond to input tapes beginning with that prefix).

Now we need to map every string to a unique color. It doesn't really matter how we do this, so just use your imagination! Going forward, just know this, each string should have its own unique color.

Now, let U_1 be some universal Turing machine. For every input program string, x , color the point $[x]$ with the color given by the output string $y = U_1(x)$. Do that for all of the (infinitely many) possible input programs, and that is our fractal!

So why is this interesting? For starters, notice that we chose U_1 arbitrarily. How might our final result have changed if we had chosen some other universal Turing machine, U_2 ? Remember the definition of universal Turing machine implies that there is some prefix string p such that $U_1(p \cdot x) = U_2(x)$ for every program x . That means if we zoom in to the interval $[p]$, the image we see will look exactly like the whole fractal if we had started with U_2 instead of U_1 ! There is nothing special about U_2 ; the fractal for *every* universal Turing machine is contained in this structure! It can be said that for a given alphabet, every one of the Oracle's fractals contains all the others!

This structure is indeed strongly fractal. We know that this structure is rough on all scales, because it would not be able to accommodate the infinite number of possible computations any other way. As for containing copies of itself, let p_{ij} denote a prefix that makes universal Turing machine U_i behave like U_j . Now from any universal Turing machine in $\{U_2, U_3, \dots\}$, we have that $U_1(p_{1j} \cdot p_{j1} \cdot x) = U_j(p_{j1} \cdot x) = U_1(x)$ for every input program string x . This means that, for every $j = 2, 3, \dots$, the interval $[p_{1j} \cdot p_{j1}]$ will be a copy of the whole fractal!

I hope this beautiful object exists in your imagination now.